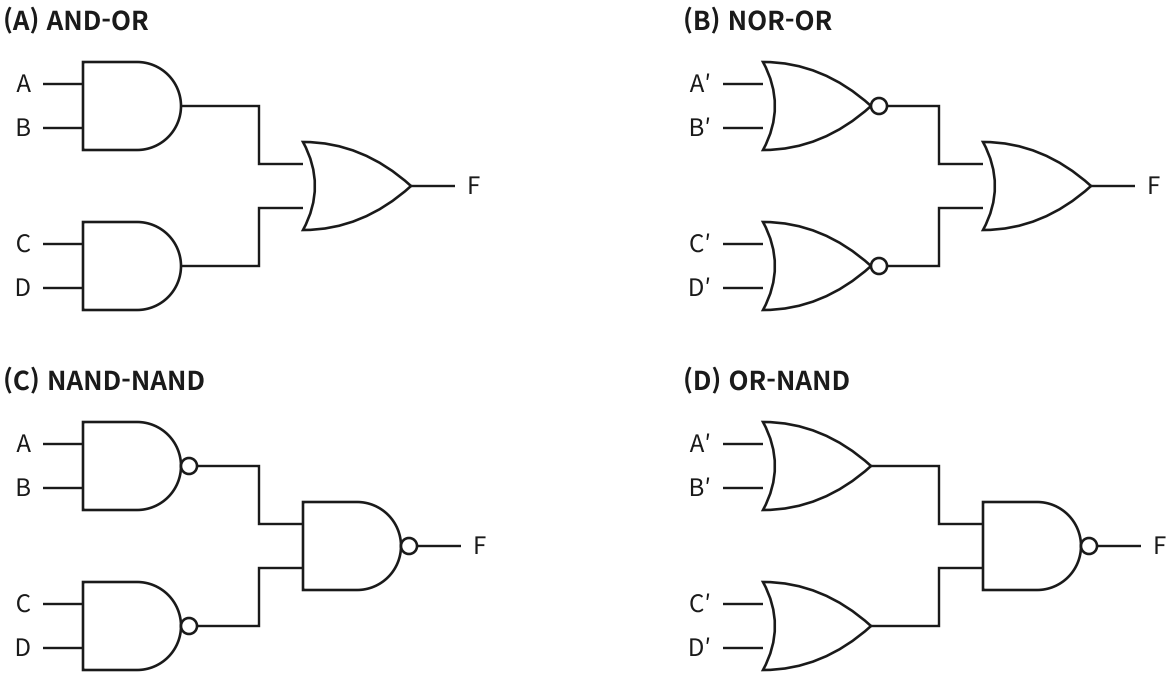
L06. Multilevel Logic

Lecture 06

6.1 SoP / PoS 를 다른 게이트로 구현하기

SoP(Sum of Products)는 곱항들을 OR로 합치는 형태이므로 기본 2단 구현은 AND-OR이고, PoS(Product of Sums)는 합항들을 AND로 곱하는 형태이므로 기본 2단 구현은 OR-AND다. 예를 들어 SoP F = AB + CD는 AND-OR 또는 NAND-NAND로 구현할 수 있으며, 입력의 보수 신호가 제공되면 NOR-OR 및 OR-NAND로도 구현할 수 있다. PoS P = (A+B)(C+D)는 OR-AND로 구현하거나, 드모르간 법칙 P = ((A+B)′ + (C+D)′)′을 이용해 NOR-NOR로 구현할 수 있다. 게이트 조합을 변경할 때는 입력과 출력의 보수 관계를 함께 확인해야 한다.

AND-OR 형태 SoP 네트워크를 다른 게이트 조합으로 바꿔 보기. 예: F = AB + CD.



F = AB + CD 를 네 가지 2단 회로로 구현한 것. 네 회로는 모두 같은 함수다.

(A) AND-OR	기본형.
(B) NOR-OR	AB = (A′ + B′)′ 이므로 AND 를 입력 보수의 NOR 로 대체.

(C) NAND-NAND	첫 단 출력이 (AB)′, 둘째 단 NAND 에서 ((AB)′(CD)′)′ = AB + CD 가 나옴.
(D) OR-NAND	(AB)′ = A′ + B′ 이므로 NAND 를 입력 보수의 OR 로 대체.

6.2 다단 논리 (Multilevel logic)

> 논리 깊이(logic depth)는 주입력에서 출력까지의 경로에 존재하는 논리 게이트 수의 최대값이다. 여기서는 SoP/PoS의 2단 구현을 설명할 때 입력 보수 생성용 인버터는 논리 단 수에서 제외한다. 2단 논리(two-level logic)는 최대 논리 깊이가 2인 조합논리 회로이며, AND-OR, OR-AND, NAND-NAND 등 다양한 게이트 조합으로 구현할 수 있다. 다단 논리(multilevel logic)는 최대 논리 깊이가 3 이상인 회로다. 일부 입력 경로가 1단 또는 2단이더라도 다른 경로가 3단 이상이면 전체 회로는 다단 논리로 분류한다.

예

X = ADF + AEF + BDF + BEF + CDF + CEF + G (SoP, 2단 논리)

X = [(A+B+C)(D+E)F] + [G] (sum-of-products-of-sums, 다단 논리)

2단 회로에는 AND 6개(입력 3개씩)와 7-입력 OR 하나가 필요함. 다단으로 바꾸려면 factoring 을 쓰는데, 그러면 단 수는 늘어남.

왜 다단 논리인가

게이트 효율(면적)	다단이 2단보다 게이트 효율이 좋은 경우가 있음. 2단 회로는 fan-in이 커서 그렇다.
속도(지연)	다단이 더 빠른 경우도 있지만 항상은 아님.
최적화	다단은 2단보다 최적화가 어렵다.
다단 단순화 기법	factoring, decomposition, extraction. 최적의 다단 논리를 보장하는 알고리즘은 없음.

1. Factoring

Factoring은 논리식의 공통 인수를 묶어 중복 계산을 줄이고 식을 더 작은 부분식으로 표현하는 기법이다. 인수분해의 목적은 식의 리터럴 수나 구현 비용을 줄이는 것이며, 논리 깊이가 반드시 증가하는 것은 아니다. 예를 들어 AB + AC = A(B+C)는 인수분해 전후 모두 최대 2단으로 구현할 수 있다. 반면 AC + AD + BC + BD + E = (A+B)(C+D) + E는 기본 AND/OR 게이트로 구현하면 최대 2단에서 3단으로 증가한다. 따라서 인수분해 후에는 게이트 비용과 논리 깊이를 각각 확인해야 한다.

예

	식	리터럴
factoring 전	F = AC + AD + BC + BD + E	9
factoring 후	F = (A + B)(C + D) + E	5

2. Decomposition

Decomposition: 식 하나를 새 식 여러 개로 쪼개서 리터럴 수를 줄이는 것. X 와 X′ 둘 다 다룸.

예

	식	리터럴
전	F = ABC + ABD + A′C′D′ + B′C′D′	12
후	F = AB(C+D) + (A′+B′)C′D′ = AB(C+D) + (AB)′(C+D)′ = XY + X′Y′	8

새 식은 X = AB, Y = C + D. 그러면 F = XY + X′Y′, 즉 X 와 Y 의 XNOR.

3. Extraction

> Extraction: 여러 출력 함수가 같이 쓰는 공통 부분 함수를 뽑아내는 것.

예

	식	리터럴 / 게이트
전	F = (A+B)CD + E, G = (A+B)E′, H = CDE	11 리터럴, 게이트 8개 (2입력 AND/OR, 인버터 제외, 출력 간 공유 전)
후	F = XY + E, G = XE′, H = YE (X = A+B, Y = CD)	11 리터럴, 게이트 6개 (같은 기준, X와 Y 공유)

6.3 다음 강의와의 관계

다음 강의(L07)에서는 다단 회로에서 경로마다 다른 게이트 지연이 만드는 글리치와 hazard 를 다룸.